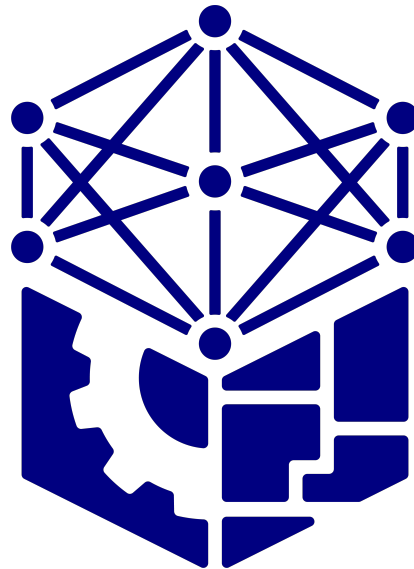


231865 200822 231442 231446

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO
FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y
MECÁNICA
ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



Informe Final – Portal Digital TUPA UNSAAC

Asignatura:

Desarrollo de Software I

Docente:

Mg. Luis Alvaro Monzón Condori

Estudiante:

Castro Pari, Rayneld Fidel
Estacio Medrano, Amilcar
Florez Vega, Yamir Wagner
Merma Ccarhuarupay, Adel Alejandro

Cusco - Perú
2026



Índice

Introducción

Antecedentes

El Texto Único de Procedimientos Administrativos (TUPA) constituye el marco normativo que concentra los procedimientos administrativos de la UNSAAC. Dentro de este marco, los estudiantes y egresados deben gestionar trámites como constancias, convalidaciones, grados y títulos, entre otros, siguiendo requisitos, costos y plazos definidos por la institución. En la práctica, la solicitud y la revisión de estos procesos están asociadas a flujos administrativos documento-centricos, donde la evidencia de pago, los requisitos y la observación del expediente se registran y validan de manera manual y distribuida entre dependencias universitarias.

Planteamiento del Problema

La gestión del TUPA en la universidad presenta una brecha entre el marco normativo institucional y la experiencia de atención digital. A partir de la documentación del proyecto y de la estructura del repositorio, el problema central se observa en la falta de trazabilidad, la dependencia de validaciones manuales y la dispersión de la información sobre requisitos, estado y observaciones de cada trámite. Esto obliga a estudiantes y personal administrativo a coordinar procesos de forma menos eficiente, con mayor riesgo mínimo de errores de seguimiento y de demora en la atención.

Justificación

El desarrollo de una plataforma digital para la gestión del TUPA permite centralizar el catálogo de procedimientos, automatizar el seguimiento del estado de cada solicitud y reducir la dependencia de documentación física, beneficiando tanto a los estudiantes solicitantes como al personal administrativo encargado de la validación.

Alcance

El alcance del presente proyecto corresponde a una **demostración funcional (demo)** del sistema, orientada a validar el flujo completo de un trámite: desde la consulta

del catálogo público, la creación y llenado de una solicitud mediante un asistente guiado, la carga de documentos y el pago, hasta la revisión y decisión por parte del personal administrativo. En la versión implementada del repositorio, el flujo de pago se modela como la carga de un comprobante de pago en el backend, pero no existe integración con un proveedor de pagos real ni con la autenticación institucional de la UNSAAC. Asimismo, la ruta `/api/auth/google` queda como un stub con respuesta 501, por lo que la autenticación con Google está fuera del alcance de la demo entregada.

Objetivos

Objetivo General

Desarrollar una plataforma web para la gestión digital de procedimientos administrativos del TUPA de la UNSAAC, que permita a los estudiantes iniciar, dar seguimiento y completar sus trámites en línea, y al personal administrativo validarlos de forma centralizada.

Objetivos Específicos

1. Diseñar un modelo de base de datos relacional que represente los procedimientos del TUPA, sus requisitos, las solicitudes de los estudiantes y su historial de seguimiento.
2. Implementar una API REST en Node.js/Express que exponga las operaciones necesarias para el catálogo de trámites, la gestión de solicitudes, la carga de documentos y la administración de usuarios.
3. Implementar una interfaz web en React que permita a los estudiantes completar un asistente de trámite paso a paso y a los administradores gestionar la cola de solicitudes pendientes.
4. Incorporar mecanismos de autenticación y autorización basados en JWT que diferencien el acceso entre el rol de estudiante y el rol administrativo.



Marco Teórico

Arquitectura Cliente-Servidor y API REST

El sistema sigue una arquitectura cliente-servidor desacoplada, en la que el cliente (frontend) y el servidor (backend) se comunican exclusivamente a través de una interfaz de programación de aplicaciones (API) que expone recursos mediante el protocolo HTTP siguiendo los principios REST (Representational State Transfer): uso de verbos HTTP (GET, POST, PUT, PATCH, DELETE) sobre recursos identificados por rutas, intercambio de datos en formato JSON y ausencia de estado en el servidor entre peticiones, delegando la sesión del usuario al token enviado en cada solicitud.

Node.js y Express

Node.js es un entorno de ejecución de JavaScript del lado del servidor basado en el motor V8, orientado a operaciones de entrada/salida no bloqueantes. Express es un framework minimalista para Node.js que provee un sistema de enrutamiento y de middlewares para construir APIs HTTP (**expressdocs**). En el presente proyecto, Express se utiliza para definir los módulos de rutas, controladores y middlewares que conforman la API REST del backend.

PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacionales de código abierto que implementa el estándar SQL con soporte para transacciones ACID, llaves foráneas, restricciones de verificación (CHECK) e índices (**postgresdocs**). El modelo de datos del sistema se apoya en estas características para garantizar la integridad referencial entre solicitudes, usuarios, trámites y documentos.

React y Vite

React es una biblioteca de JavaScript para la construcción de interfaces de usuario basada en componentes reutilizables y un modelo declarativo de renderizado (**reactdocs**). Vite es la herramienta de compilación y servidor de desarrollo utilizada para el proyecto

frontend, que ofrece recarga en caliente y un empaquetado optimizado para producción.

Autenticación basada en JSON Web Tokens (JWT)

JWT es un estándar abierto (RFC 7519) que define un formato compacto y autocontenido para transmitir información entre dos partes como un objeto JSON firmado digitalmente (**rfc7519**). En el sistema, el token JWT se emite tras un inicio de sesión exitoso y contiene el identificador del usuario y su rol, permitiendo validar cada solicitud subsecuente sin necesidad de mantener sesiones en el servidor.

Hash de Contraseñas

Para el almacenamiento seguro de credenciales, las contraseñas no se guardan en texto plano, sino como el resultado de una función de hash con salteo (*salt*) mediante el algoritmo `bcrypt`, diseñado específicamente para dificultar ataques de fuerza bruta mediante un factor de costo configurable.

Metodología de Desarrollo

El desarrollo del proyecto se organizó de forma iterativa e incremental, construyendo primero el modelo de datos, luego la capa de API y finalmente la integración con la interfaz de usuario. La evidencia verificable en el repositorio corresponde a una estructura modular de trabajo por carpetas (`extttbackend/` y `extttfrontend/`), con definición formal de rutas y servicios y con un flujo de validación local mediante ejecución del servidor y pruebas automatizadas. No se encuentra evidencia comprobable en el proyecto de un tablero Kanban o de una herramienta de gestión de sprints distinta a la organización académica y de entrega del curso, por lo que el enfoque se describe como una metodología de desarrollo ágil informal, apoyada en la implementación continua y en pruebas de integración HTTP.

Multer y gestión de archivos

La carga de documentos y comprobantes de pago se implementa en el backend mediante la librería `multer`, un middleware de Express para la recepción de archivos

multipart/form-data

parentitemulterdocs. En el repositorio, esta capa se configura con almacenamiento en disco y validación de tipo de archivo para aceptar únicamente PDF, JPG y PNG, con límite de tamaño de 5 MB por carga.

Desarrollo

Descripción General del Sistema

El sistema, denominado Portal Digital TUPA - UNSAAC, está compuesto por tres capas: una aplicación cliente construida en React que consume una API REST desarrollada en Node.js/Express, la cual a su vez persiste la información en una base de datos relacional PostgreSQL. El sistema contempla dos tipos de usuario con necesidades y permisos distintos.

Usuarios y Roles

- **Estudiante** (tabla `usuario_general`): puede consultar el catálogo público de trámites, iniciar una solicitud mediante un asistente guiado de varios pasos, cargar los documentos requeridos, realizar el pago, dar seguimiento a sus expedientes y recibir notificaciones sobre el estado de estos.
- **Administrador** (tabla `usuario_admin`): puede revisar la cola de solicitudes pendientes, validar o rechazar los documentos cargados, registrar observaciones y decisiones sobre cada expediente, y gestionar el catálogo de trámites y los usuarios del sistema. Un administrador puede tener el rol ADMIN o SUPER_ADMIN.

Requerimientos

Requerimientos Funcionales

La tabla anterior se valida directamente con las rutas expuestas por el backend (`exttt/api/procedures`, `exttt/api/requests`, `exttt/api/admin`) y con los componentes de navegación definidos en `extttfrontend/src/App.jsx`. El único componente que queda fuera del alcance de la demo es la integración con un proveedor de pagos real y la autenticación de Google, ambos presentados como stubs o funcionalidades no implementadas en la

Cuadro 1: Requerimientos funcionales del sistema y su estado de implementación.

Código	Descripción	Estado
RF-01	Consulta pública del catálogo de trámites, con filtros por búsqueda, categoría, costo y plazos.	Implementado
RF-02	Inicio de sesión de estudiantes y administradores mediante credenciales del sistema.	Implementado
RF-03	Creación de una solicitud en borrador y avance del wizard por pasos 1 a 6.	Implementado
RF-04	Carga de comprobante de pago y documentos requeridos por requisito.	Implementado en demo
RF-05	Generación automática de número de expediente al confirmar el envío.	Implementado
RF-06	Rastreo público por número de expediente.	Implementado
RF-07	Notificaciones de estado y observaciones para el usuario.	Implementado
RF-08	Cola administrativa de solicitudes pendientes con revisión por expediente.	Implementado
RF-09	Decisión administrativa sobre validación de documentos y estado de la solicitud.	Implementado
RF-10	Gestión del catálogo y activación/desactivación de usuarios y trámites.	Implementado

versión entregada.

Requerimientos No Funcionales

Arquitectura del Sistema

Arquitectura General

La Figura ?? presenta la arquitectura general del sistema, organizada en tres capas desacopladas que se comunican mediante peticiones HTTP/JSON.

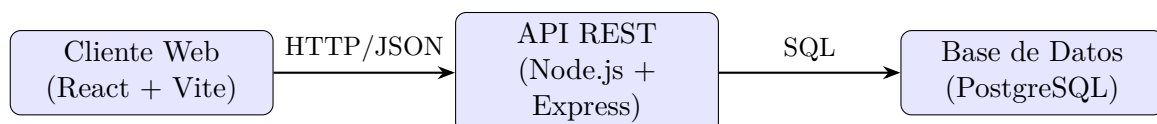


Figura 1: Arquitectura general del sistema en tres capas.

Arquitectura del Backend

La organización real del backend en el repositorio es consistente con un patrón Rutas–Controladores–Servicios, pero además incorpora utilidades y miembros adicionales

Cuadro 2: Requerimientos no funcionales del sistema.

Código	Descripción
RNF-01	Las contraseñas de los usuarios deben almacenarse cifradas mediante una función de hash (<code>bcrypt</code>), nunca en texto plano.
RNF-02	El acceso a los recursos protegidos de la API debe requerir un token JWT válido en la cabecera <code>Authorization</code> .
RNF-03	Las rutas administrativas deben restringirse mediante verificación de rol (<code>admin</code> / <code>super_admin</code>).
RNF-04	Todas las consultas a la base de datos deben usar sentencias parametrizadas para prevenir inyección SQL.
RNF-05	La interfaz debe ser consumible desde un navegador web moderno, comunicándose con el backend mediante peticiones HTTP habilitadas por CORS.

de infraestructura:

- `routes/`: define los prefijos de la API (`/api/auth`, `/api/users`, `/api/procedures`, `/api/requests`, `/api/documents`, `/api/notifications` y `/api/admin`) y enlaza cada ruta con su controlador.
- `controllers/`: reciben la solicitud HTTP, delegan la lógica a los servicios y responden con JSON.
- `services/`: contienen la lógica de negocio y las consultas parametrizadas sobre PostgreSQL, incluidas la creación de solicitudes, la generación del expediente, la validación documental y la gestión de usuarios y trámites.
- `middleware/`: implementa la autenticación JWT, autorización por rol, manejo de errores y carga de archivos mediante `multer`.
- `db/`: centraliza el *pool* de conexiones a PostgreSQL.
- `utils/`: contiene utilidades como la generación de el número de expediente y la firma/verificación de tokens JWT.

Arquitectura del Frontend

La organización real del frontend presenta una división funcional por tipo de vista, pero con una estructura más detallada que la descrita de forma general:

- `pages/public`: páginas de acceso público como landing, catálogo, búsqueda y rastreo de expedientes.

- `pages/auth`: flujo de autenticación.
- `pages/student`: panel del estudiante, perfil, notificaciones, trámites propios y detalle de solicitudes.
- `pages/student/wizard`: asistentes del flujo de solicitud en seis pasos.
- `pages/admin`: panel administrativo, cola de pendientes, validación documental, gestión de procedimientos y usuarios.
- `components/`, `context/`, `hooks/`, `lib/` y `styles/`: capas de reutilización de interfaz, estado global de sesión y estilos compartidos.

Modelo de Base de Datos

Diccionario de Datos

El modelo relacional está compuesto por once entidades. La Tabla ?? resume su propósito dentro del sistema.

Cuadro 3: Diccionario de datos del modelo relacional.

Entidad	Descripción
especialidad	Catálogo de especialidades/facultades a las que puede pertenecer un estudiante.
usuario_general	Estudiantes que solicitan trámites; incluye datos institucionales y de contacto.
usuario_admin	Personal administrativo del sistema, con rol ADMIN o SUPER_ADMIN.
categoria	Agrupación temática de los trámites del catálogo (p. ej. Certificados, Matrícula).
tramite	Catálogo oficial de procedimientos del TUPA: costo, plazo, base legal y vigencia.
requisito	Requisitos documentales obligatorios u opcionales asociados a cada trámite.
solicitud	Expediente creado por un estudiante para un trámite: estado, prioridad, número de expediente.
documento	Archivo digital cargado por el estudiante para cumplir un requisito específico.
observacion	Observación registrada por un administrador sobre un documento o solicitud.
seguimiento	Bitácora histórica de cambios de estado de una solicitud.
notificacion	Alerta generada hacia un estudiante ante un evento relevante de su solicitud.

Relaciones Principales

Un `usuario_general` puede tener múltiples solicitud (una por trámite iniciado); cada solicitud referencia un trámite del catálogo. Cada trámite define uno o más requisito, y cada documento cargado por el estudiante se asocia opcionalmente a un requisito específico dentro de una solicitud. Las tablas `observacion` y `seguimiento` registran, respectivamente, las incidencias y el historial de cambios de estado de una solicitud, ambas vinculadas al `usuario_admin` que las generó.

Restricciones de Integridad

El modelo aplica restricciones CHECK para limitar los valores válidos de los campos de estado, evitando así estados inconsistentes a nivel de base de datos:

- `solicitud.estado` $\in$ {BORRADOR, SOLICITADO, VERIFICANDO_PAGO, PAGADO, EN PROCESO, SUBSANACION, OBSERVADO, COMPLETADO, ANULADO, RECHAZADO}.
- `documento.estado_validacion` $\in$ {PENDIENTE, APROBADO, RECHAZADO}.
- `observacion.estado` $\in$ {PENDIENTE, SUBSANADO, CERRADO}.

El script de creación completo se incluye en el Anexo ??.

Diseño de la API REST

La API se organiza en siete módulos funcionales, cada uno bajo su propio prefijo de ruta. La Tabla ?? resume los módulos implementados y su conjunto de operaciones definido en la capa de rutas del backend.

Diseño de Interfaces

El frontend implementado en React y Vite expone rutas para los principales escenarios del sistema. A partir de la estructura de navegación en `frontend/src/App.jsx` y de los archivos presentes en `frontend/src/pages/`, la interfaz está organizada en los siguientes bloques:

Cuadro 4: Endpoints principales de la API REST del backend.

Módulo	Ruta	Endpoints implementados
Autenticación	/api/auth	POST /login, POST /google (stub no implementado).
Perfil de usuario	/api/users	GET /profile, PUT /profile, PUT /profile/password.
Catálogo de trámites	/api/procedures	GET /, GET /categories, GET /:cod extunderscore tramite.
Solicitudes	/api/requests	POST /, GET /, GET /:id, PATCH /:id/step, POST /:id/voucher, POST /:id/document/:id extunderscore requisito, POST /:id/submit, GET /track/:numero extunderscore expediente.
Documentos	/api/documents	DELETE /:id extunderscore documento.
Notificaciones	/api/notifications	POST /read-all, PATCH /:id/read.
Administración	/api/admin	GET /stats, GET /requests, GET /requests/:id, POST /requests/:id/decision, POST /procedures, PATCH /procedures/:cod extunderscore tramite/toggle, GET /users, PATCH /users/:id/toggle.

Portal Público

La navegación pública incluye la landing page, el catálogo de trámites, la búsqueda de expediente y la vista de seguimiento público a través de los componentes `LandingPage.jsx`, `Catalog.jsx`, `TrackingSearch.jsx` y `TrackingResults.jsx`. Estas vistas coinciden con el acceso descrito por las rutas `/`, `/catalogo`, `/seguimiento` y `/seguimiento/resultados`.

Portal del Estudiante

El portal del estudiante incorpora el dashboard principal, el perfil, el centro de notificaciones, el listado de trámites y solicitudes, y el wizard de compra y envío de documentos bajo los pasos 1 a 6. En la estructura del frontend, estos módulos se materializan en `Dashboard.jsx`, `MyProfile.jsx`, `Notifications.jsx`, `MyProcedures.jsx`, `MyRequests.jsx`, `RequestDetail.jsx` y las vistas de wizard/`Step1.jsx` a `Step6.jsx`.

Portal Administrativo

El portal administrativo implementado incluye el dashboard de gestión, la cola de solicitudes pendientes, la validación documental, la gestión del catálogo de procedimientos y la administración de usuarios. Estas vistas se encuentran en `AdminDashboard.jsx`, `PendingQueue.jsx`, `DocumentValidation.jsx`, `ProcedureManagement.jsx` y `UserManagement.jsx` dentro de `frontend/src/pages/admin/`.

Seguridad

El sistema implementa los siguientes mecanismos de seguridad a nivel de API:

- **Autenticación:** un middleware `authenticateToken` intercepta las rutas protegidas, valida el token JWT recibido en la cabecera `Authorization: Bearer <token>` y adjunta la identidad del usuario a la petición.
- **Autorización por rol:** un middleware `requireRole` restringe el acceso a las rutas bajo `/api/admin` únicamente a usuarios con rol `admin` o `super_admin`.

- **Cifrado de contraseñas:** las contraseñas se almacenan como *hash* mediante `bcrypt`, nunca en texto plano.
- **Prevención de inyección SQL:** todas las consultas a la base de datos se ejecutan mediante sentencias parametrizadas.
- **CORS:** el backend habilita el origen del frontend de desarrollo (`http://localhost:5173`) para permitir las peticiones entre dominios distintos durante el desarrollo.

Despliegue

La versión disponible en el repositorio se ejecuta en entorno local para la presentación de la demo. El backend corre con Node.js y Express sobre el puerto 3000, mientras que el frontend se sirve con Vite sobre `localhost:5173`. Las variables de entorno necesarias se registran en `.env` del backend y incluyen la cadena de conexión a PostgreSQL, `JWT_SECRET`, el puerto HTTP y las credenciales de acceso a la base de datos. No existe evidencia de un despliegue productivo en un servicio en la nube o en un host externo; por lo tanto, el despliegue verificado es local y orientado a demostración académica.

Pruebas

La verificación del sistema se realizó con pruebas automatizadas de integración HTTP mediante Jest y Supertest sobre la aplicación Express. En la ejecución real del repositorio, la suite reportó **4 suites** y **30 tests** correctos, cubriendo el health check, login, perfil, catálogo, wizard de solicitudes, gestión documental, notificaciones y endpoints administrativos. Además, el recorrido de prueba de humo validado en el flujo del sistema incluye la creación de un borrador de solicitud, la actualización del proceso en pasos, la carga de comprobante y documentos, y el envío final con generación de expediente. Estos resultados confirman que la API expuesta por el backend responde con los códigos y mensajes esperados en los escenarios principales de la demo.

Conclusiones

Resultados

Diseño del modelo de base de datos relacional: _____.

Implementación de la API REST: _____.

Implementación de la interfaz web (asistente de trámite y panel administrativo):
_____.

Mecanismos de autenticación y autorización: _____.

Conclusiones

1. El uso de una arquitectura desacoplada entre el frontend en React y el backend en Express permitió desarrollar y probar ambas capas de forma independiente, comunicándose únicamente a través de contratos de API bien definidos.
2. El modelo de datos relacional en PostgreSQL, con restricciones CHECK sobre los campos de estado, permitió delegar parte de la validación del flujo del trámite a la propia base de datos, reduciendo la posibilidad de estados inconsistentes.

Recomendaciones

1. Para una versión posterior al alcance de demo, se recomienda incorporar pruebas automatizadas (unitarias y de integración) sobre los servicios de la API, particularmente sobre la lógica de cambio de estado de una solicitud.
2. Se recomienda evaluar la integración con un proveedor de pagos real y con el sistema de autenticación institucional de la UNSAAC para una eventual puesta en producción.